5. Online Speech-to-Text (ASR)

- **5. Online Speech-to-Text (ASR)**
  - 1. Introduction
    - 1.1 What is “ASR”?
    - 1.2 Implementation Principles
      - 1. **Acoustic Model**
      - 2. **Language Model**
      - 3. **Pronunciation Dictionary**
      - 4. **Decoder**
      - 5. **End-to-End ASR**
  - 2. Project Architecture
    - Key Code
      - 1. Speech Processing and Recognition Core (`largemodel/largemodel/asr.py`)
      - 2. VAD Smart Recording (`largemodel/largemodel/asr.py`)
    - Code Analysis
  - 3. Practical Operation
    - 3.1 Configuring Online ASR
    - 3.2 Launching and Testing the Feature

1. Introduction

1.1 What is “ASR”?

ASR (Automatic Speech Recognition) is a technology that converts human speech signals into text. It is widely used in intelligent assistants, voice command control, automated telephone customer service, real-time caption generation, and other fields. The goal of ASR is to enable machines to “understand” human language and convert it into a form that computers can process and understand.

1.2 Implementation Principles

The implementation of an ASR system mainly relies on the following key technical components:

1. Acoustic Model

- The acoustic model is responsible for converting the input sound signal into phonemes or sub-word units. This typically involves feature extraction steps, such as using Mel-frequency cepstral coefficients (MFCCs) or filter banks to represent the audio signal.
- These features are then fed into deep neural networks (DNNs), convolutional neural networks (CNNs), recurrent neural networks (RNNs), or more advanced Transformer architectures for training to learn how to map audio features to corresponding phonemes or characters.

2. Language Model

- A language model is used to predict the next most likely word given context, thus helping to improve recognition accuracy. It is trained on large amounts of text data to understand which word sequences are more likely to occur.
- Common language models include n-gram models, RNN-based LMs, and the recently popular Transformer-based LMs.

3. Pronunciation Dictionary

- A pronunciation dictionary provides a mapping between words and their corresponding pronunciations. This is crucial for connecting the acoustic model and the language model, as it allows the system to understand and match heard sounds based on known pronunciation rules.

4. Decoder

- The decoder's task is to find the most likely word sequence as output, given the acoustic model, the language model, and the pronunciation dictionary. This process typically involves complex search algorithms, such as the Viterbi algorithm or graph-based search methods, to find the optimal path.

5. End-to-End ASR

- With the development of deep learning, end-to-end ASR systems have emerged. These systems attempt to learn text output directly from raw audio signals without requiring explicit separate designs for acoustic models, pronunciation dictionaries, and language models. These systems are often based on sequence-to-sequence (Seq2Seq) frameworks, such as using attention mechanisms or Transformer architectures, greatly simplifying the complexity of traditional ASR systems.

In summary, modern ASR systems achieve efficient and accurate human speech-to-text conversion by combining the above components and training with large-scale datasets and powerful computing resources. With technological advancements, the performance of ASR systems continues to improve, and their application scenarios are becoming increasingly widespread.

2. Project Architecture

Key Code

1. Speech Processing and Recognition Core (`largemodel/largemodel/asr.py`)

```
# From largemodel/largemodel/asr.py
def kws_handler(self)->None:
    if self.stop_event.is_set():
        return

    if self.listen_for_speech(self.mic_index):
        asr_text = self.ASR_conversion(self.user_speechdir)  # 进行 ASR 转换 / Perform ASR conversion
        if asr_text =='error':  # 检查 ASR 结果长度是否小于4个字符 / Check if ASR result length is less than 4 characters
            self.get_logger().warn("I still don't understand what you mean. Please try again")
            playsound(self.audio_dict[self.error_response])  # 错误响应 / Error response
        else:
            self.get_logger().info(asr_text)
            self.get_logger().info("okay😜, let me think for a moment...")
            self.asr_pub_result(asr_text)  # 发布 ASR结果 / Publish ASR result

    else:
```

```
        return

def ASR_conversion(self, input_file:str)->str:
    if self.use_oline_asr:
        result=self.modelinterface.oline_asr(input_file)
        if result[0] == 'ok' and len(result[1]) > 4:
            return result[1]
        else:
            self.get_logger().error(f'ASR Error:{result[1]}')  # ASR error.
            return 'error'
    else:
        result=self.modelinterface.SenseVoiceSmall_ASR(input_file)
        if result[0] == 'ok' and len(result[1]) > 4:
            return result[1]
        else:
            self.get_logger().error(f'ASR Error:{result[1]}')  # ASR error.
            return 'error'
```

2. VAD Smart Recording (`largemodel/largemodel/asr.py`)

```
# From largemodel/largemodel/asr.py
def listen_for_speech(self,mic_index=0):
    p = pyaudio.PyAudio()   # Create PyAudio instance. / 创建PyAudio实例。
    audio_buffer = []       # Store audio data. / 存储音频数据。
    silence_counter = 0     # Silence counter. / 静音计数器。
    MAX_SILENCE_FRAMES = 90  # 30帧*30ms=900ms静音后停止 / Stop after 900ms of
silence (30 frames * 30ms)
    speaking = False  # Flag indicating speech activity. / 语音活动标志。
    frame_counter = 0  # Frame counter. / 计数器。
    stream_kwargs = {
        'format': pyaudio.paInt16,
        'channels': 1,
        'rate': self.sample_rate,
        'input': True,
        'frames_per_buffer': self.frame_bytes,
    }
    if mic_index != 0:
        stream_kwargs['input_device_index'] = mic_index

    # Prompt the user to speak via the buzzer. / 通过蜂鸣器提示用户讲话。
    self.pub_beep.publish(UInt16(data = 1))
    time.sleep(0.5)
    self.pub_beep.publish(UInt16(data = 0))

    try:
        # Open audio stream. / 打开音频流。
        stream = p.open(**stream_kwargs)
        while True:
            if self.stop_event.is_set():
                return False

            frame = stream.read(self.frame_bytes, exception_on_overflow=False)
 # Read audio data. / 读取音频数据。
            is_speech = self.vad.is_speech(frame, self.sample_rate)  # VAD
detection. / VAD检测。

            if is_speech:
```

```
                # Detected speech activity. / 检测到语音活动。
                speaking = True
                audio_buffer.append(frame)
                silence_counter = 0
            else:
                if speaking:
                    # Detect silence after speech activity. / 在语音活动后检测静音。
                    silence_counter += 1
                    audio_buffer.append(frame)  # Continue recording buffer. / 持续记录缓冲。

                    # End recording when silence duration meets the threshold. / 静音持续时间达标时结束录音。
                    if silence_counter >= MAX_SILENCE_FRAMES:
                        break
            frame_counter += 1
            if frame_counter % 2 == 0:
                self.get_logger().info('1' if is_speech else '-')
                # Real-time status display.
    finally:
        stream.stop_stream()
        stream.close()
        p.terminate()

    # Save valid recording (remove trailing silence). / 保存有效录音（去除尾部静音）。
    if speaking and len(audio_buffer) > 0:
        # Trim the last silent part. / 裁剪最后静音部分。
        clean_buffer = audio_buffer[:-MAX_SILENCE_FRAMES] if len(audio_buffer) > MAX_SILENCE_FRAMES else audio_buffer

        with wave.open(self.user_speechdir, 'wb') as wf:
            wf.setnchannels(1)
            wf.setsampwidth(p.get_sample_size(pyaudio.paInt16))
            wf.setframerate(self.sample_rate)
            wf.writeframes(b''.join(clean_buffer))
            return True
```

Code Analysis

The ASR (Speech-to-Text) functionality is provided by the `ASRNode` node (`asr.py`). This node is responsible for audio recording, conversion, and publishing.

1. **Audio Recording (`listen_for_speech`):**

- This function uses the `pyaudio` library to capture an audio stream from the microphone.
- It integrates the `webrtcvad` library for Voice Activity Detection (VAD). The function loops through audio frames and uses `vad.is_speech()` to determine if each frame contains human voice.
- When speech is detected, data is written to a buffer. Recording stops when sustained silence (defined by `MAX_SILENCE_FRAMES`) is detected.
- Finally, the audio data in the buffer is written to a `.wav` file located at `self.user_speechdir`.

2. **Backend Selection and Execution (`ASR_conversion`):**

- The `kws_handler` function calls the `ASR_conversion` function after successful recording.

- This function determines which backend implementation to call by reading the ROS parameter `use_oline_asr` (a boolean value).
- If `false`, the `self.modelinterface.SenseVoiceSmall_ASR` method is called, corresponding to local recognition.
- If `true`, the `self.modelinterface.oline_asr` method is called, corresponding to online recognition.
- This function passes the audio file path as an argument to the selected method and processes the returned result.

3. **Result Publication (`asr_pub_result`):**

- After `ASR_conversion` returns valid text, `kws_handler` calls the `asr_pub_result` function.
- This function wraps a text string in a `std_msgs.msg.String` message and publishes it to the `/asr` topic via the ROS publisher.

3. Practical Operation

3.1 Configuring Online ASR

1. **First, obtain the API Key from the registration tutorial in the previous Tongyi Qianwen.**
2. **Open the model interface configuration file `large_model_interface.yaml`:**

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

3. **Enter your API credentials**:

Find the online ASR section, enter your Key, and then select Speech Recognition to Model.

```
# large_model_interface.yaml
# ------------------------------------------------------------------------------
# 大模型设置 (Large Language Models Settings)
# ------------------------------------------------------------------------------
## 在线大模型 (Online Large Language Models)
# 通义千问平台配置 (Tongyi Platform Configuration)
online_asr_api_key: "Your_AccessKey_ID"

# ------------------------------------------------------------------------------
# 语音识别设置 (Speech Recognition Settings)
# ------------------------------------------------------------------------------
## 在线语音识别 (Online ASR)
oline_asr_model: 'paraformer-realtime-8k-v2'  # Optional: 'paraformer-realtime-
v2', 'paraformer-realtime-v1', etc., using the API of the Tongyi Qianwen
platform.
oline_asr_sample_rate: 16000      # ASR audio sampling rate
```

4. **Open the main configuration file `yahboom.yaml`:**

```
gedit ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

5. **Enable online ASR mode:**

Set the `use_oline_asr` parameter to `True`.

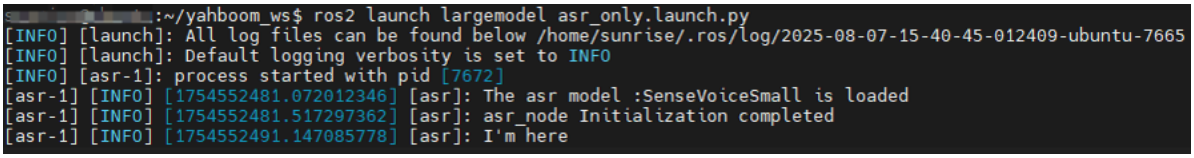
```yaml
# yahboom.yaml

asr:
  ros__parameters:
    use_oline_asr: True  # Key: Change from False to True
    regional_setting : "China"
```

3.2 Launching and Testing the Feature

1. **Launch Command**:

```
ros2 launch largemodel asr_only.launch.py
```

2. **Test**:

Speak into the microphone: "Hi,yahboom" and it will respond: "I'm here," and then you can start speaking. Finally, it will display the recorded audio converted into text and printed in the terminal.